

Parte 1 - Investigación sobre el patrón Page Object Model y responde:

1. ¿Qué es Page Object Model?

El modelo de objetos de página (POM en inglés) es un patrón de diseño utilizado en pruebas de automatización. Es una manera de ordenar los elementos y acciones de una página web en clases y objetos separados. Esto ayuda a que el código sea más fácil de mantener y probar y a su vez reduce la duplicación de código.

Tomado de

https://www.selenium.dev/documentation/test_practices/encouraged/page_object_models/

2. ¿Cuál es el objetivo principal de este patrón?

El objetivo principal está en hacer más sencillas las pruebas de automatización mediante la organización de elementos y acciones en clases separadas, lo que hace que haya una separación entre la lógica de testeo y la estructura de UI.

3. ¿Qué problemas intenta resolver?

- Primeramente los scripts de pruebas no tan sólidos que se rompan fácilmente con cambios de UI.
- Después el código duplicado que se puede ir acumulando.
- Finalmente la dificultad para adaptarse a cambios en la interfaz, lo que hace que el desarrollo sea más lento.

4. ¿Cuáles son sus principales características?

- Clases de página (Page Classes) : Clase dedicada por página la cual tiene los métodos y localizadores.
- Localizadores (Locators): Forma en la que se identifica un elemento en la página centralizados en la clase.
- Métodos / Acciones: funciones que ejecutan interacciones del usuario que las pruebas invocan en lugar de hacerlo con los elementos.
- Separación de responsabilidades : Acciones -> Objetos de página / Aserciones (assertions) -> Scripts de prueba (no en clases de página)
- Composición para elementos reutilizables: Un componente tipo barra de navegación que se repite en varias páginas, se puede modelar como "Page Component Object" independiente.
- Interfaz fluida: Los métodos de navegación retornan el objeto de la página siguiente lo que permite encadenar llamadas.

5. ¿Qué ventajas ofrece respecto a tener las pruebas sin POM?

Lo que se ha podido compartir anteriormente: reutilización de código y escalabilidad en múltiples pruebas, mantenimiento sencillo, mejor legibilidad, menos duplicación de código y mejor abstracción y encapsulamiento.

6. ¿Cuales son algunas desventajas o limitaciones?

Lo de la regla de >una clase por página< no es tan útil en aplicaciones reales ya que componentes como navbars o footers se deben manejar como componentes reutilizables aparte ya que si no se hace así se termina duplicando código.

POM no es un framework, siempre va a necesitar un test runner (TestNG, JUnit) y en un driver (Selenium, Playwright).

Se debe siempre respetar la separación de responsabilidades sino se perdería la capacidad de reutilización

7. La relación con POM de:

Principio de Responsabilidad Única

- Cada clase se encarga únicamente de representar esa página. Las aserciones se dejan por fuera para darle una responsabilidad clara sin mezclar responsabilidades extra.

Reutilización de código

- Los métodos definidos en una clase de página pueden reutilizar en múltiples pruebas.

Mantenibilidad

- A centralizar los localizadores y acciones en clases separadas por página, solo ameritaría un cambio en un archivo único si se hiciera alguna alteración en la UI. Esto reduce el trabajo de mantenimiento.

Desde la pregunta 2, tomado de:

<https://www.testmuai.com/learning-hub/page-object-model/>

Parte 2 - Investigación sobre TDD y responda:

Esta parte fue hecha completamente por IA y se utilizó ChatGPT con el siguiente resultado:

1. ¿Qué es TDD?

Test Driven Development (TDD) es una metodología de desarrollo de software en la que las pruebas se escriben antes de implementar la funcionalidad que se quiere construir.

La idea principal es definir primero cuál debe ser el comportamiento esperado del software y convertir esa expectativa en una prueba automatizada. Después se implementa únicamente el código necesario para lograr que esa prueba pase.

TDD normalmente se representa mediante el ciclo **Red - Green - Refactor**:

- **Red:** se escribe una prueba que representa el comportamiento esperado. La prueba falla porque la funcionalidad todavía no existe o no cumple con ese comportamiento.
- **Green:** se implementa el código mínimo necesario para hacer que la prueba pase.
- **Refactor:** una vez que la funcionalidad funciona correctamente, se mejora la estructura interna del código sin cambiar su comportamiento.

Una forma sencilla de entenderlo es que en la etapa Red se define mediante una prueba qué queremos que haga el sistema. En Green se programa la solución hasta obtener el resultado esperado, y en Refactor se mejora el "cómo" sin modificar el "qué".

Por esta razón, TDD no es únicamente una técnica para encontrar errores. También funciona como una forma de diseñar software a partir de comportamientos concretos y verificables.

2. ¿Cuáles son las ventajas de TDD?

Una de las principales ventajas de TDD es que obliga al desarrollador a pensar primero en el comportamiento esperado antes de concentrarse en la implementación.

Esto ayuda a reducir ambigüedades porque antes de escribir el código se debe tener claro qué resultado se espera obtener.

Otra ventaja es que favorece la creación de código modular. Si una función, clase o componente debe probarse de manera aislada, normalmente es necesario mantener responsabilidades claras y evitar dependencias innecesarias.

TDD también crea una red de seguridad para realizar cambios. Cuando una funcionalidad ya cuenta con pruebas automatizadas, estas pueden ejecutarse nuevamente después de modificar o refactorizar el código para comprobar que el comportamiento anterior continúa funcionando.

Entre sus principales ventajas se pueden mencionar:

- Permite detectar errores durante etapas tempranas del desarrollo.
- Ayuda a definir claramente el comportamiento esperado.
- Favorece código modular y con responsabilidades más claras.
- Facilita futuras refactorizaciones.
- Reduce el riesgo de introducir regresiones.
- Genera pruebas automatizadas como parte natural del desarrollo.
- Aumenta la confianza al modificar funcionalidades existentes.
- Facilita el trabajo en equipo al dejar ejemplos ejecutables del comportamiento esperado.

3. ¿Cuáles son las limitaciones de TDD?

TDD también presenta limitaciones y no debe considerarse una solución automática para todos los problemas de software.

Una de sus principales dificultades es que requiere disciplina y experiencia. Al inicio puede sentirse más lento, porque antes de implementar una funcionalidad se debe pensar en escenarios de prueba y escribirlos de forma adecuada.

Además, una prueba puede estar técnicamente correcta pero representar mal el requerimiento. Si se define incorrectamente el comportamiento esperado, se puede obtener una prueba en estado Green aunque la solución no responda correctamente a la necesidad real del sistema.

Otra limitación es que TDD puede resultar más complejo en escenarios donde existen muchas dependencias externas, interfaces visuales altamente cambiantes o procesos donde todavía se está explorando cuál será la solución final.

También es importante entender que tener muchas pruebas no significa automáticamente tener software de calidad. Las pruebas deben representar casos relevantes y estar diseñadas correctamente.

Algunas limitaciones son:

- Requiere tiempo inicial para diseñar las pruebas.
- Necesita disciplina por parte del equipo.
- No garantiza que los requisitos estén correctamente definidos.
- Una mala prueba puede validar un comportamiento incorrecto.
- Puede ser difícil de aplicar en ciertas integraciones o interfaces muy cambiantes.
- No sustituye pruebas de integración, pruebas de interfaz ni validaciones manuales cuando estas son necesarias.

4. ¿En qué se diferencia TDD de escribir pruebas después de implementar el código?

La principal diferencia se encuentra en el momento en que se crean las pruebas y en el papel que estas tienen durante el desarrollo.

En TDD, la prueba se escribe primero y funciona como una especificación del comportamiento que todavía debe implementarse.

El proceso se puede representar así:

Requerimiento → Prueba → RED → Implementación → GREEN → REFACTOR

Cuando las pruebas se escriben después de desarrollar la funcionalidad, el proceso normalmente ocurre así:

Requerimiento → Implementación → Prueba → Validación

Las dos estrategias pueden producir pruebas útiles, pero existe una diferencia importante.

En TDD, las pruebas pueden influir directamente en la forma en que se diseña el código desde el inicio. En cambio, cuando se agregan después, la arquitectura y las decisiones principales de implementación ya existen y las pruebas se utilizan principalmente para verificar lo construido.

Por ejemplo, encontrar pruebas automatizadas dentro de un proyecto no significa automáticamente que ese proyecto haya sido desarrollado utilizando TDD. Para demostrar TDD tendría que existir evidencia de que las pruebas fueron escritas antes de la implementación y que se siguió el ciclo Red-Green-Refactor.

5. ¿Cómo contribuye TDD a la calidad del software?

TDD contribuye a la calidad del software porque introduce ciclos cortos de validación durante el proceso de desarrollo.

En lugar de construir una funcionalidad completa y descubrir errores al final, se trabaja mediante pequeños comportamientos comprobables. Cada prueba permite verificar que una parte específica del sistema funciona como se espera.

También favorece la mantenibilidad, porque un código diseñado para poder probarse normalmente tiende a tener responsabilidades más claras y menor acoplamiento.

Otro beneficio importante aparece durante la refactorización. Si el comportamiento ya está protegido por pruebas, es posible modificar la estructura interna del código y volver a ejecutar los tests para comprobar que los resultados siguen siendo correctos.

Esto permite distinguir entre dos aspectos:

- **Qué hace el software:** comportamiento esperado.
- **Cómo está construido:** implementación interna.

Durante un refactor se puede modificar el "cómo", mientras las pruebas ayudan a comprobar que el "qué" permanece igual.

TDD también puede mejorar el trabajo en equipo, porque las pruebas sirven como ejemplos ejecutables de reglas y comportamientos importantes del sistema.

En resumen, TDD puede contribuir a:

- Detectar defectos de forma temprana.
- Evitar regresiones.
- Mejorar la mantenibilidad.
- Facilitar refactorizaciones.
- Promover componentes más pequeños y comprobables.
- Documentar comportamientos importantes mediante pruebas.
- Aumentar la confianza del equipo al realizar cambios.

Relación con el proyecto EcoWash

En EcoWash existen actualmente pruebas automatizadas y una estrategia de Page Object Model implementada en el trabajo relacionado con EC-40.

Sin embargo, durante la revisión del historial del proyecto no se encontró evidencia suficiente para afirmar que esa implementación haya sido desarrollada utilizando TDD.

El componente `ContactForm` y su lógica ya existían antes de que se agregaran las pruebas correspondientes. Posteriormente se incorporaron las pruebas, la configuración de Vitest y la estrategia POM.

Por esta razón, en este trabajo se diferencia entre:

- **Tener pruebas automatizadas, y**
- **Haber desarrollado utilizando TDD.**

Esta diferencia es importante porque TDD no se determina únicamente observando que las pruebas actuales están en estado exitoso, sino analizando el proceso mediante el cual fueron creadas.

Parte 3 - Análisis de Proyecto

Para esta parte se utilizó inteligencia artificial para completar el requisito completamente. El modelo usado fue Sonnet 5 de Claude. El prompt fue el siguiente:

Necesito que seas un experto desarrollador senior y estes haciendo un analisis y pruebas generales sobre un login ficticio de una pagina de una empresa de limpieza de tapiceria con nombre Ecowash. Necesito hacer las pruebas sin POM y luego con POM para detectar y analizar como funciona realmente. Dame tambien ejemplos con fragmentos de código para entenderlo de una manera mas clara.

Asumo Playwright con TypeScript como herramienta de pruebas E2E, por ser el estándar más común en proyectos Next.js:

1. Pruebas generadas (ejemplo ilustrativo: login de Ecowash)

Escenario ficticio: un usuario ingresa a `ecowash.com/login`, escribe su correo y contraseña, presiona "Iniciar sesión" y debe llegar a su panel (`/dashboard`) viendo el mensaje "Bienvenido de nuevo".

2. Implementación SIN POM

Sin el patrón, el script de prueba interactúa directamente con el DOM: localizadores y acciones viven mezclados dentro del propio test.

```
// tests/login.spec.ts (SIN POM)
import { test, expect } from '@playwright/test';

test('el usuario puede iniciar sesión correctamente', async ({ page }) => {
  await page.goto('https://ecowash.com/login');
  await page.locator('#email').fill('cliente@ecowash.com');
  await page.locator('#password').fill('Clave123!');
  await page.locator('button[type="submit"]').click();

  await expect(page.locator('h1')).toHaveText('Bienvenido de nuevo');
  await expect(page).toHaveURL(/.*\s/dashboard/);
});

test('muestra error con credenciales inválidas', async ({ page }) => {
  await page.goto('https://ecowash.com/login');

  await page.locator('#email').fill('cliente@ecowash.com');
  await page.locator('#password').fill('claveIncorrecta');
  await page.locator('button[type="submit"]').click();

  await expect(page.locator('.error-message')).toHaveText('Credenciales incorrectas');
});
```

- Problema: si mañana el equipo de frontend cambia #email por #input-correo, o el botón pasa a tener data-testid="login-btn", hay que ir prueba por prueba (aquí son 2, pero en un proyecto real pueden ser decenas) buscando y reemplazando el localizador.

3. Implementación CON POM

Se crea una clase LoginPage que encapsula los localizadores y las acciones. El test solo describe el escenario, no la mecánica de cómo interactuar con el DOM.

```
// pages/LoginPage.ts
import { Page, Locator } from '@playwright/test';
export class LoginPage {
  readonly page: Page;
  readonly emailInput: Locator;
  readonly passwordInput: Locator;
  readonly loginButton: Locator;
  readonly errorMessage: Locator;

  constructor(page: Page) {
    this.page = page;
    this.emailInput = page.locator('#email');
    this.passwordInput = page.locator('#password');
    this.loginButton = page.locator('button[type="submit"]');
    this.errorMessage = page.locator('.error-message');
  }
}
```

```
async goto() {
  await this.page.goto('https://ecowash.com/login');
}
async login(email: string, password: string) {
  await this.emailInput.fill(email);
  await this.passwordInput.fill(password);
  await this.loginButton.click();
}
}
```

```
// pages/DashboardPage.ts
import { Page, Locator } from '@playwright/test';

export class DashboardPage {
  readonly page: Page;
  readonly welcomeTitle: Locator;
  constructor(page: Page) {
    this.page = page;
  }
}
```

```
this.welcomeTitle = page.locator('h1');
}
```

```
// tests/login.spec.ts (CON POM)
import { test, expect } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';
import { DashboardPage } from '../pages/DashboardPage';

test('el usuario puede iniciar sesión correctamente', async ({ page }) => {
  const loginPage = new LoginPage(page);
  const dashboardPage = new DashboardPage(page);

  await loginPage.goto();
  await loginPage.login('cliente@ecowash.com', 'Clave123!');
  await expect(dashboardPage.welcomeTitle).toHaveText('Bienvenido de nuevo');
  await expect(page).toHaveURL(/.*\s/dashboard/);
});

test('muestra error con credenciales inválidas', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.goto();
  await loginPage.login('cliente@ecowash.com', 'claveIncorrecta');

  await expect(loginPage.errorMessage).toHaveText('Credenciales
incorrectas');
});
```

Ahora, si #email cambia a #input-correo, solo se edita una línea dentro de LoginPage.ts. Ninguna prueba se toca.

4. Comparación con y sin POM

Aspecto	Sin POM	Con POM
Ubicación de localizadores	Repetidos dentro de cada test	Centralizados en <code>LoginPage.ts</code>
Cambio de un <code>id</code> en el HTML	Se edita en cada test que lo usa	Se edita en un solo archivo
Legibilidad del test	Mezcla "cómo" interactuar con "qué" se prueba	El test solo dice <code>loginPage.login(...)</code> , se lee casi como texto natural
Reutilización	Cada test reescribe la secuencia de llenar campos y dar clic	<code>login()</code> se reutiliza en pruebas de login exitoso, fallido, bloqueo de cuenta, etc.
Escalabilidad	Se vuelve inmanejable al crecer la suite (ej. agregar pruebas de recuperación de contraseña, registro, etc.)	Cada página nueva agrega su propia clase sin afectar las existentes

Aspecto	Sin POM	Con POM
Riesgo de duplicación	Alto: la secuencia "llenar campos + clic" se copia y pega en cada archivo de prueba	Bajo: un solo método encapsula la acción

¿Por qué la versión con POM es más mantenible?

En el ejemplo sin POM, el localizador #email aparece en cada prueba que necesite loguearse (login exitoso, fallido, pruebas de otras funcionalidades de Ecowash que primero requieren autenticarse, etc.). Un cambio de diseño en el formulario de login obliga a tocar todos esos archivos. Con POM, esa responsabilidad vive en un solo lugar (LoginPage), así que el cambio se hace una vez y todas las pruebas que dependen de ella se benefician automáticamente — es justamente el argumento de mantenibilidad que vimos en la Parte I, aplicado a un caso concreto de Ecowash.

Parte 4 - Reflexiones

1. ¿Qué aprendimos sobre POM?

El valor de organizar bien todo para reducir costos reales de mantenimiento si algo tuviera que actualizarse. Algo que ayudó mucho fue el ejemplo creado con IA para tener mejor entendimiento de lo que esta pasando cuando se hacen las pruebas y el impacto real en un proyecto de gran escala. Al final centraliza toda la logica de pruebas y se hace mas sencillo ejecutarlo.

2. ¿ QUÉ aprendimos sobre TDD?

TDD puede entenderse como una forma de convertir primero una expectativa en una prueba y utilizar esa prueba para guiar la implementación.

El ciclo Red-Green-Refactor permite trabajar de manera incremental:

- **Red:** definir mediante una prueba lo que se espera obtener.
- **Green:** implementar la solución mínima necesaria para alcanzar ese resultado.
- **Refactor:** mejorar la forma en que está construida la solución sin modificar el comportamiento conseguido.

Lo más importante es entender que TDD no busca generar errores intencionalmente, sino definir primero una condición verificable antes de construir la solución que debe satisfacerla.

3. ¿ Los agentes y skills de IA favorecen la mantenibilidad?

Lastimosamente aún no se han podido hacer código de pruebas con el proyecto Ecowash generado por agentes de IA. Aun así, con base en lo investigado hasta el momento sobre el POM y TDD si los agentes generan pruebas siguiendo este patrón, esto podría favorecer aún más la mantenibilidad del proyecto.

4. ¿ Qué mejoras propondrían?

Podria ser generar localizadores estables en lugar de IDs que cambian con el diseño. Otra podria ser separar siempre componentes reutilizables (como las barras de navegación en Ecowash)